

# Transforms

**DongJoon Kim**

| Phone : +82-2-940-8461

| Email : [dongjoonkim@kw.ac.kr](mailto:dongjoonkim@kw.ac.kr)

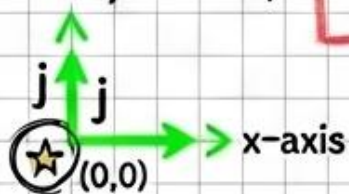
# Beginner's Guide to Transform

스케치 노트로 배우는 Step-by-Step 튜토리얼

## 핵심 개념

- 선형 변환 (Linear Transform)
  - 행렬(Matrix)로 표현,
  - 원점 유지, 직선 유지
- 어파인 변환 (Affine Transform)
  - 선형 변환 + 이동(Translation)

좌표계 기저  
(Coordinate System Basis)



회전 (Rotation)

$$R(v) = R * v$$

이동 (Translation)

$$T(v) = v + t$$

## 유용한 팁

- 순서가 중요!  
 $R * S * v \neq S * R * v$
- 기저 벡터 (Basis Vectors)
- 좌표계 (Coordinate System)의 기준

크기 조정 (Scaling)

$$S(v) = S * v$$

이동 (Translation)

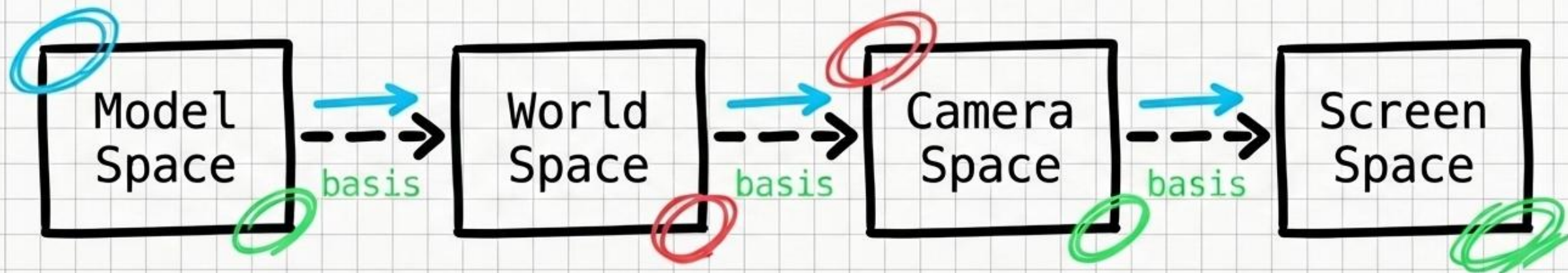
$$T(v) = v + t$$

Transform = Translation(Scaling(Rotation(v)))

복합 변환 예시

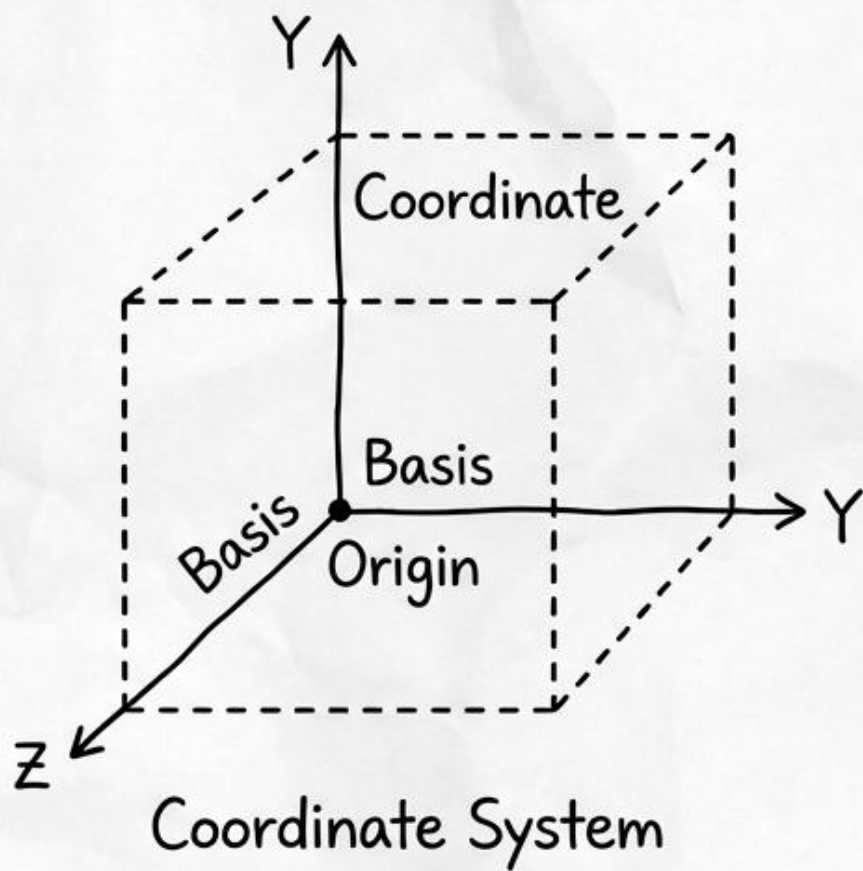


# 그래픽스의 목적지



그래픽스의 핵심은 물체를 원하는 Space로 이동시키는 것!  
Model Space에서 시작해 Screen Space로 가는  
이 모든 여정을 Graphics Transformation Pipeline이라고 부릅니다.

# 세계를 정의하는 단어들



- ✓ Space: 공간 그 자체.
- ✓ Coordinate System: 위치를 좌표로 표현하는 수학적 시스템 (원점과 Basis로 구성).
- ✓ Frame: 공간에 구체적으로 놓인 Coordinate System (즉 좌표계 인스턴스)



# 공간의 두 가지 기본 요소

## Point

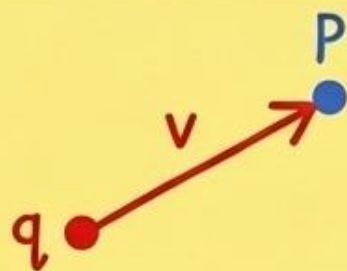
공간 안의 고정된 '위치'  
(Position).

## Vector

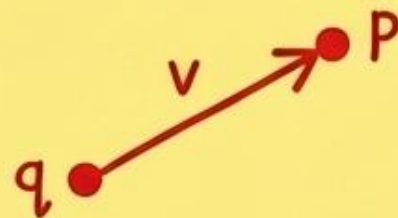
위치와 무관한 '방향과 크기'  
(Direction & Magnitude).



# Point 와 Vector 의 수학 법칙



Point - Point = Vector  
(두 점 사이의 이동 거리와 방향)

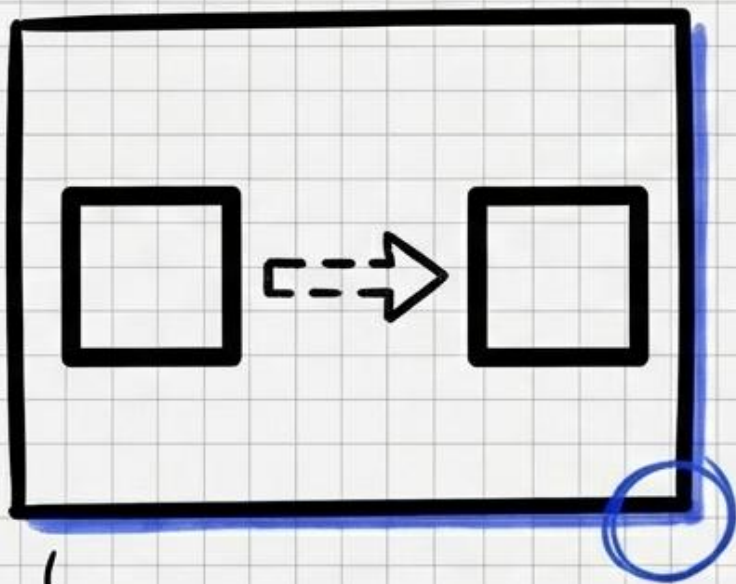


Point + Vector = Point  
(점에서 특정 방향으로 이동한 새로운 위치)

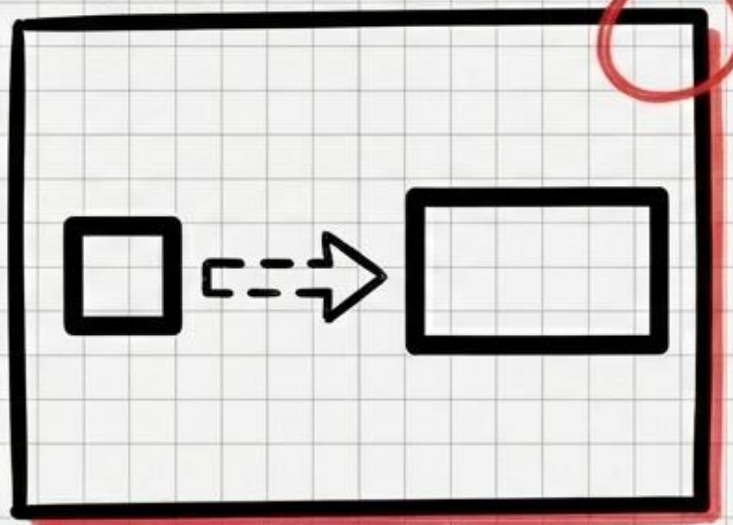
**\*주의: Point끼리 더하거나 스케일링하는 것은 의미가 없는 연산입니다!**

# 물체를 움직이는 3가지 기본기

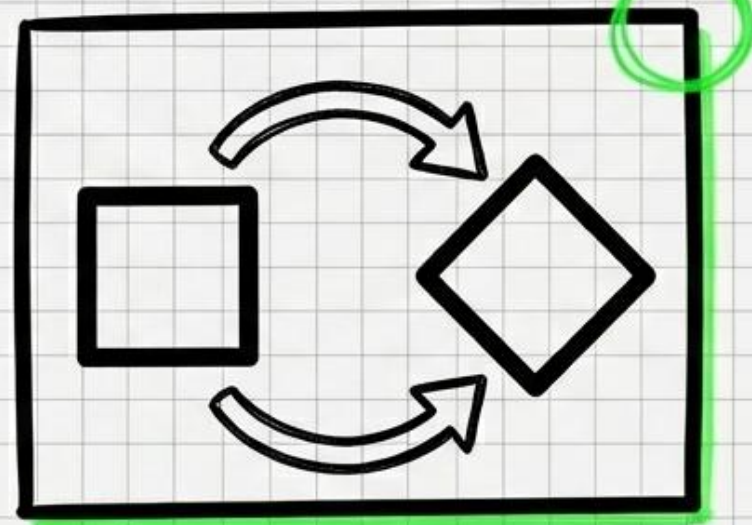
이제 물체를 2D 공간에서 조작하는 세 가지 기본 Transform을 차례대로 알아봅시다.



→ Translation



→ Scaling

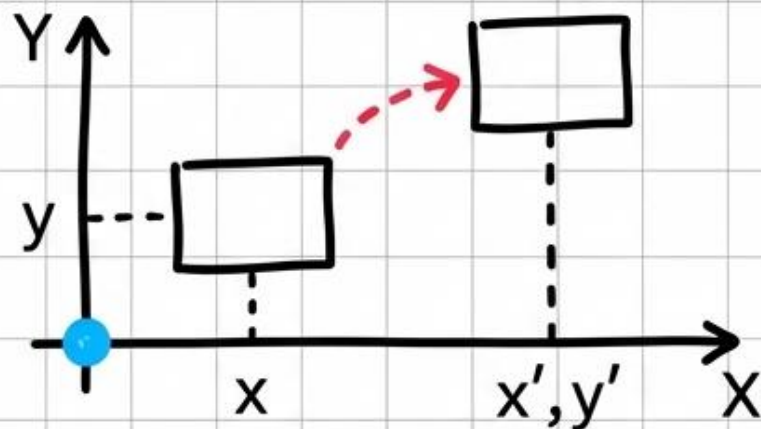


→ Rotation



# Move & Stretch

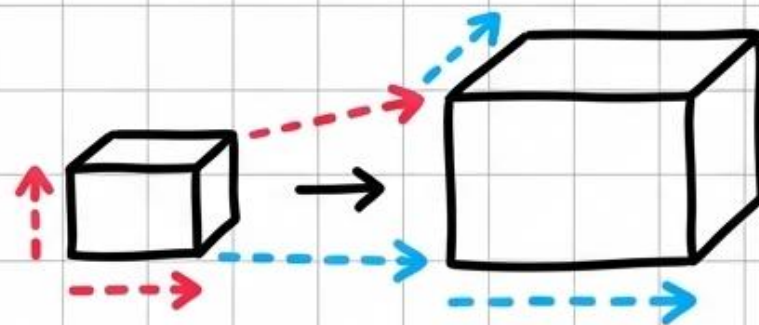
## Translation



$$\begin{aligned}x' &= x + tx \\ y' &= y + ty\end{aligned}$$

(더하기 연산)

## Scaling

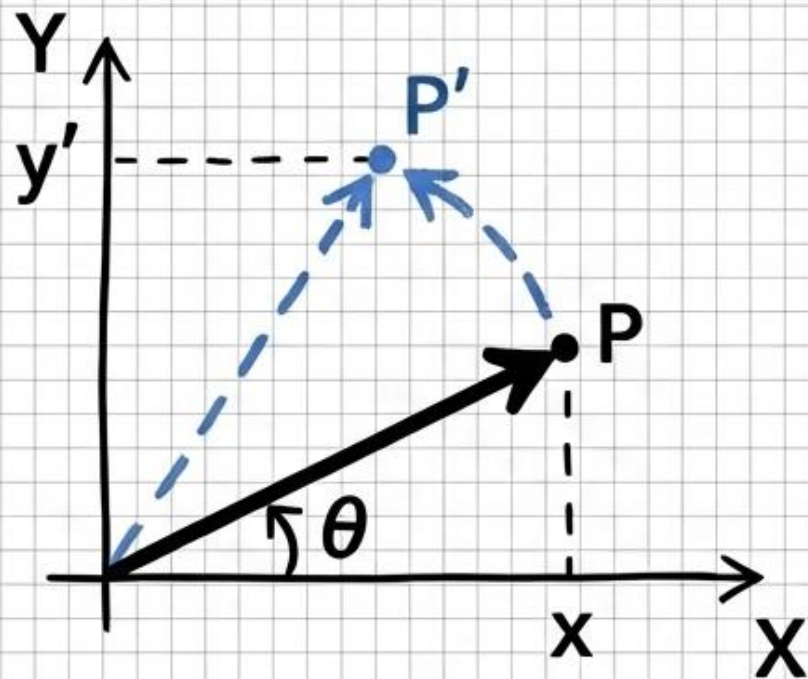


$$S(s) = \begin{pmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

(곱하기 연산)



# Rotation



원점을 기준으로  
회전합니다.

$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

복잡해 보이지만, 삼각함수 매트릭스  
곱셈(Matrix Multiplication) 하나로  
깔끔하게 표현됩니다.

# 골치 아픈 문제 (Euclidean Transform)

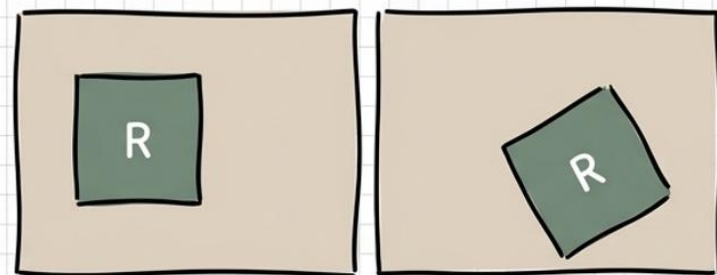

$$\begin{bmatrix} \text{Rotation} \\ \text{Matrix} \end{bmatrix} * \begin{bmatrix} \text{Point} \end{bmatrix} + \begin{bmatrix} \text{Translation} \\ \text{Vector} \end{bmatrix} = ???$$

Red arrows and question marks highlight the ambiguity of the operations: a curved arrow from the matrix to the point with a red asterisk, and a curved arrow from the point to the vector with a red plus sign.

\* Rotation과 Scaling은 곱셈(\*)

\* Translation은 덧셈(+)

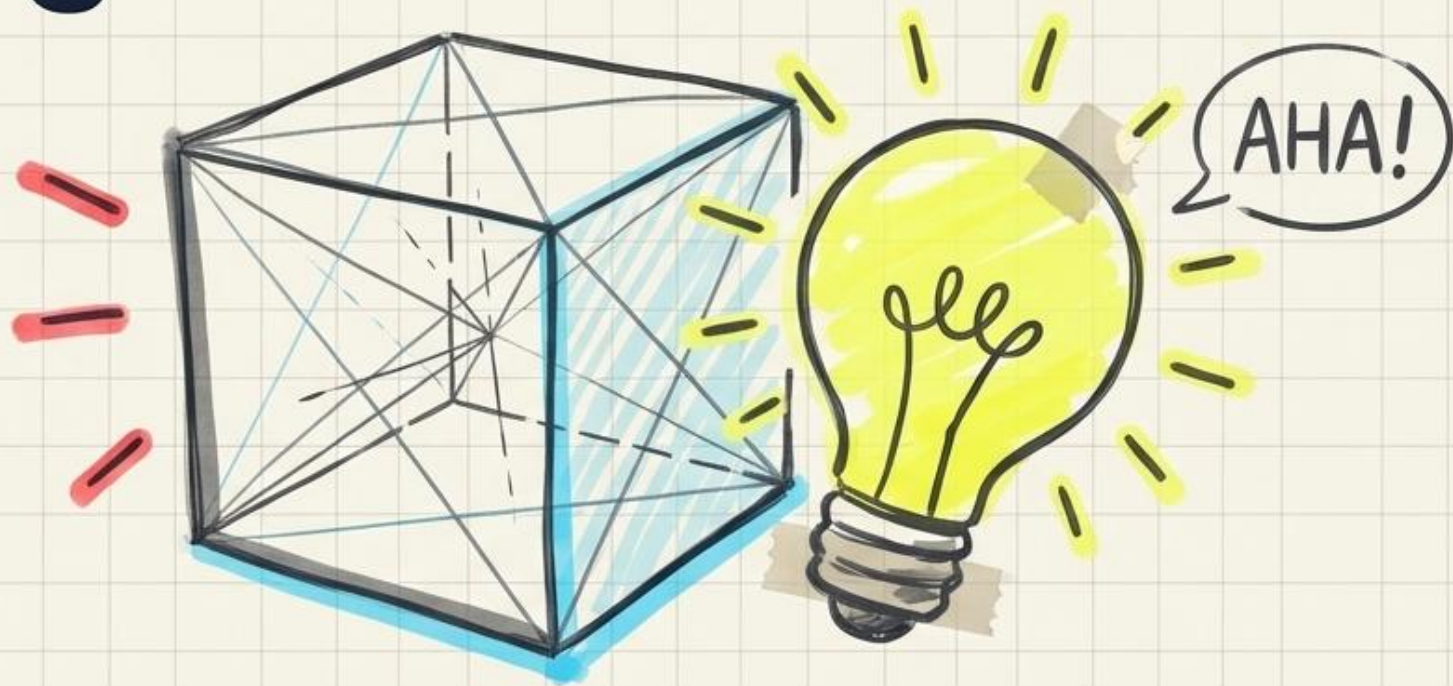
연산 방식이 달라 한 번에 계산할 수 없고,  
역산(Inverse)도 복잡해집니다.



3DOF (1 rotation, 2 translation)



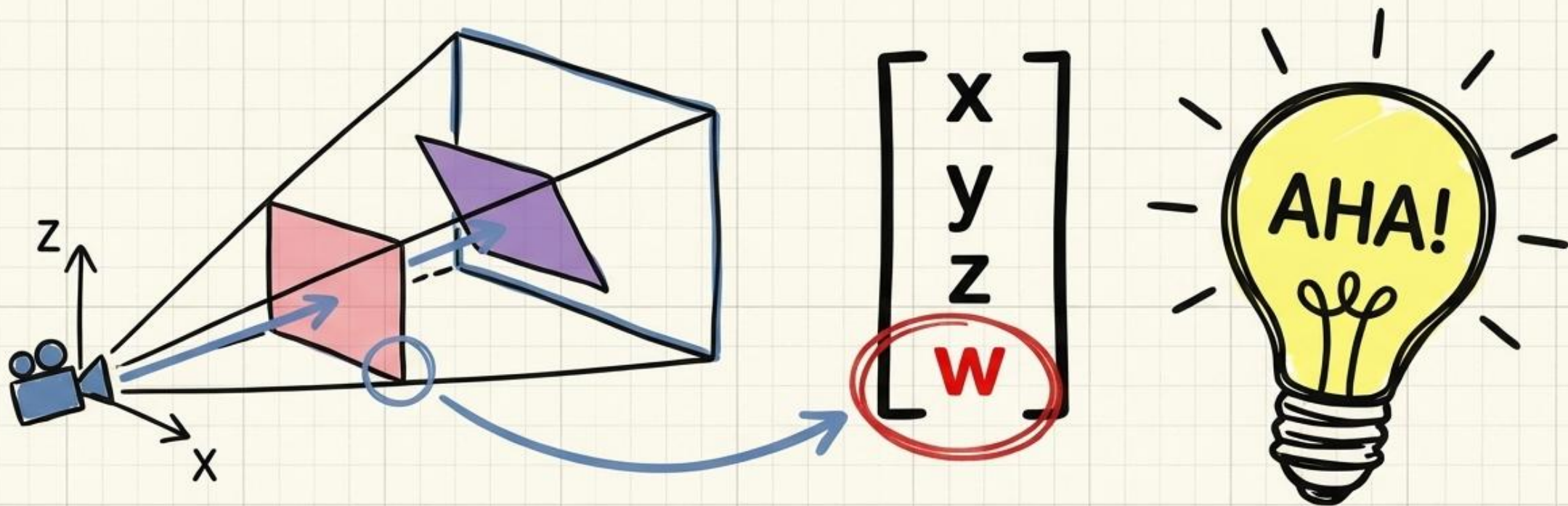
# 궁극의 해결책: Homogeneous Coordinates



복잡한 3D 변환을 한 방에 해결하는 수학적 마법



# 해결책은 ‘차원을 하나 추가’하는 것입니다!



좌표에  $(w)$ 를 하나 더 붙이면, 이동·회전·크기변환을 모두 같은 행렬곱으로 처리할 수 있습니다.

# w의 법칙: Point vs Vector



Point (점): 위치를 가지므로 이동의 영향을 받음 ->  $w = 1$   $[x, y, z, 1]$

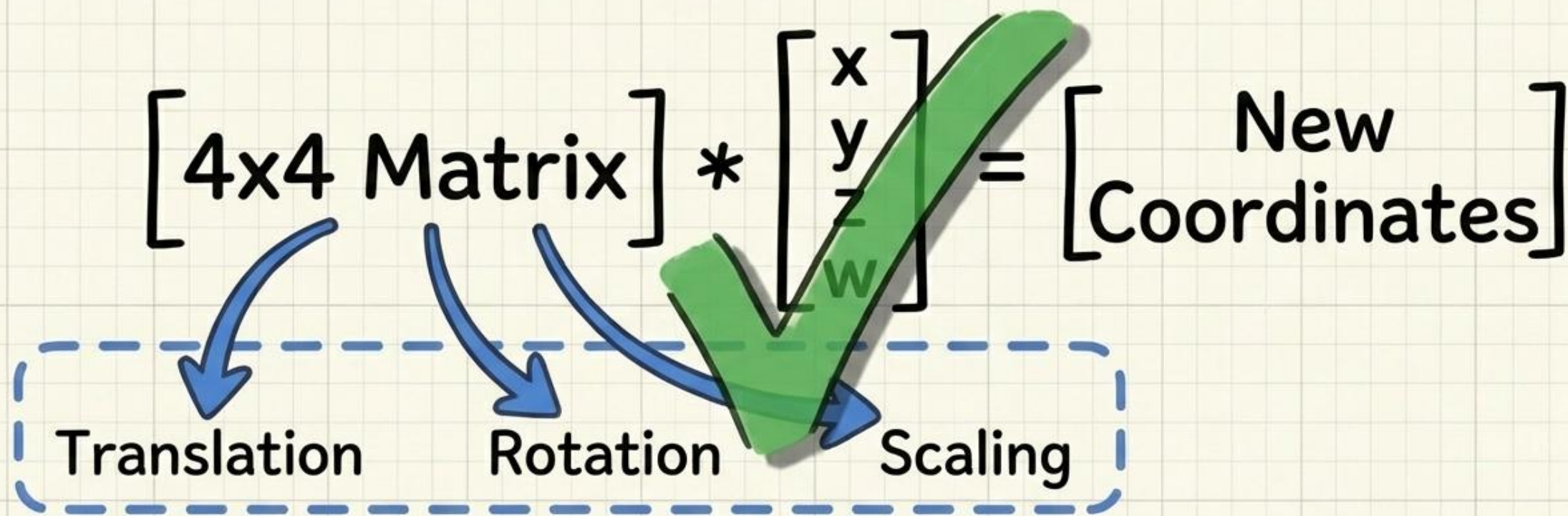


Vector (벡터): 방향만 가지므로 이동의 영향을 안 받음 ->  $w = 0$   $[x, y, z, 0]$

“(w)는 점과 벡터를 구분하는 식별자 역할을 합니다.”



# 마법의 결과: 모든 것을 하나의 행렬로!



이제 (w) 덕분에 덧셈 없이, 모든 변환을 하나의 4x4 행렬곱으로 통일해서 다룰 수 있습니다.





## 오해 금지! (Misconception Warning)

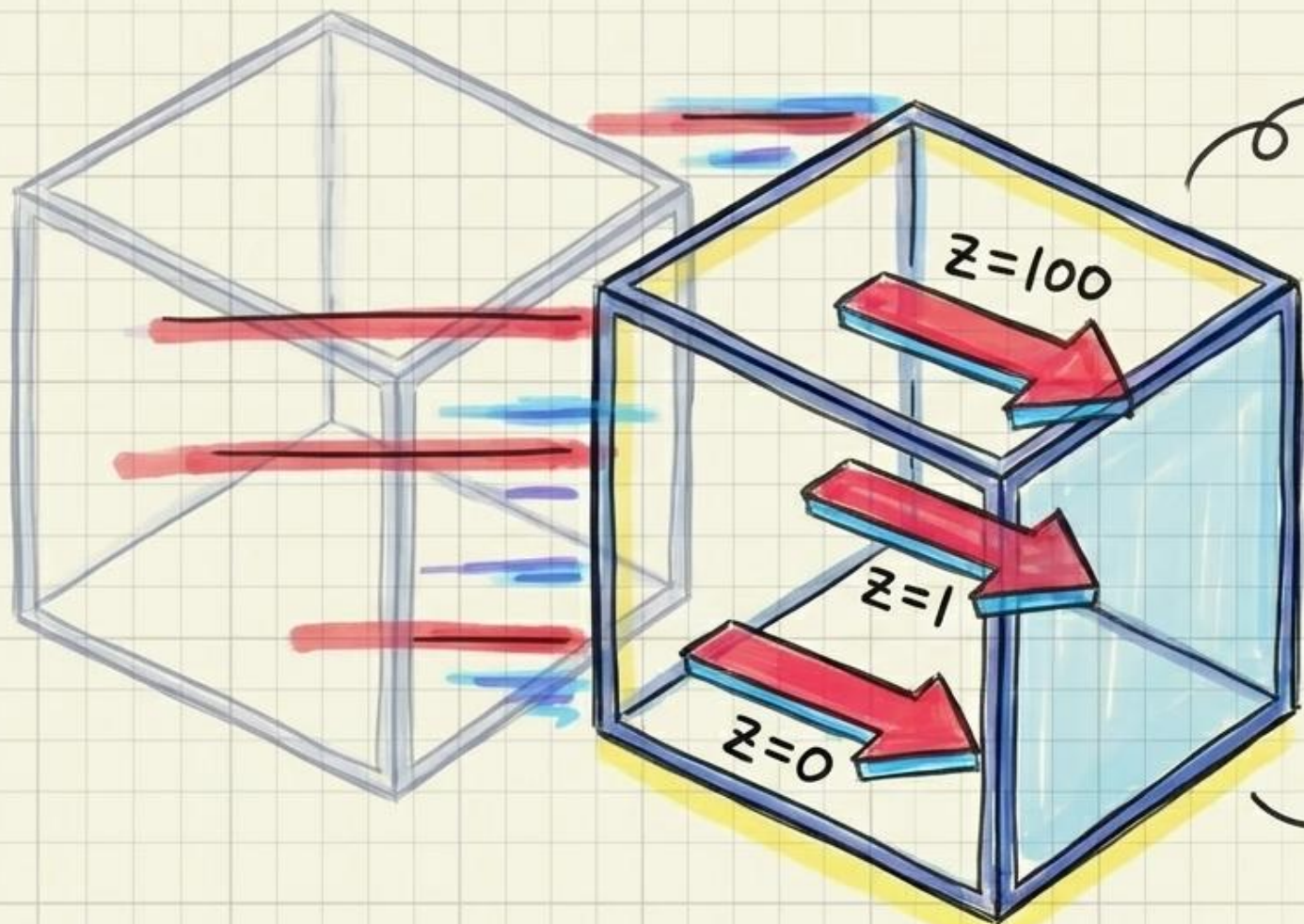
~~2D에  $w$ 를 추가해 진짜 3D가 되거나, 3D에  $w$ 를 추가해 진짜 4차원 공간으로 가는 것이 아닙니다.~~



3D 점은  $[x, y, z, 1]$ , 3D 벡터는  $[x, y, z, 0]$   
0처럼 ( $w$ )를 추가해 표현할 뿐입니다.



# 4x4 Translation: 방 전체를 통째로 밀기



\* Rigid motion (강체 운동)

\* 3D 유클리드 공간 전체의 균일한 이동.

\*  $z=0, z=1, z=100$   
어디서든 이동량이 동일합니다.

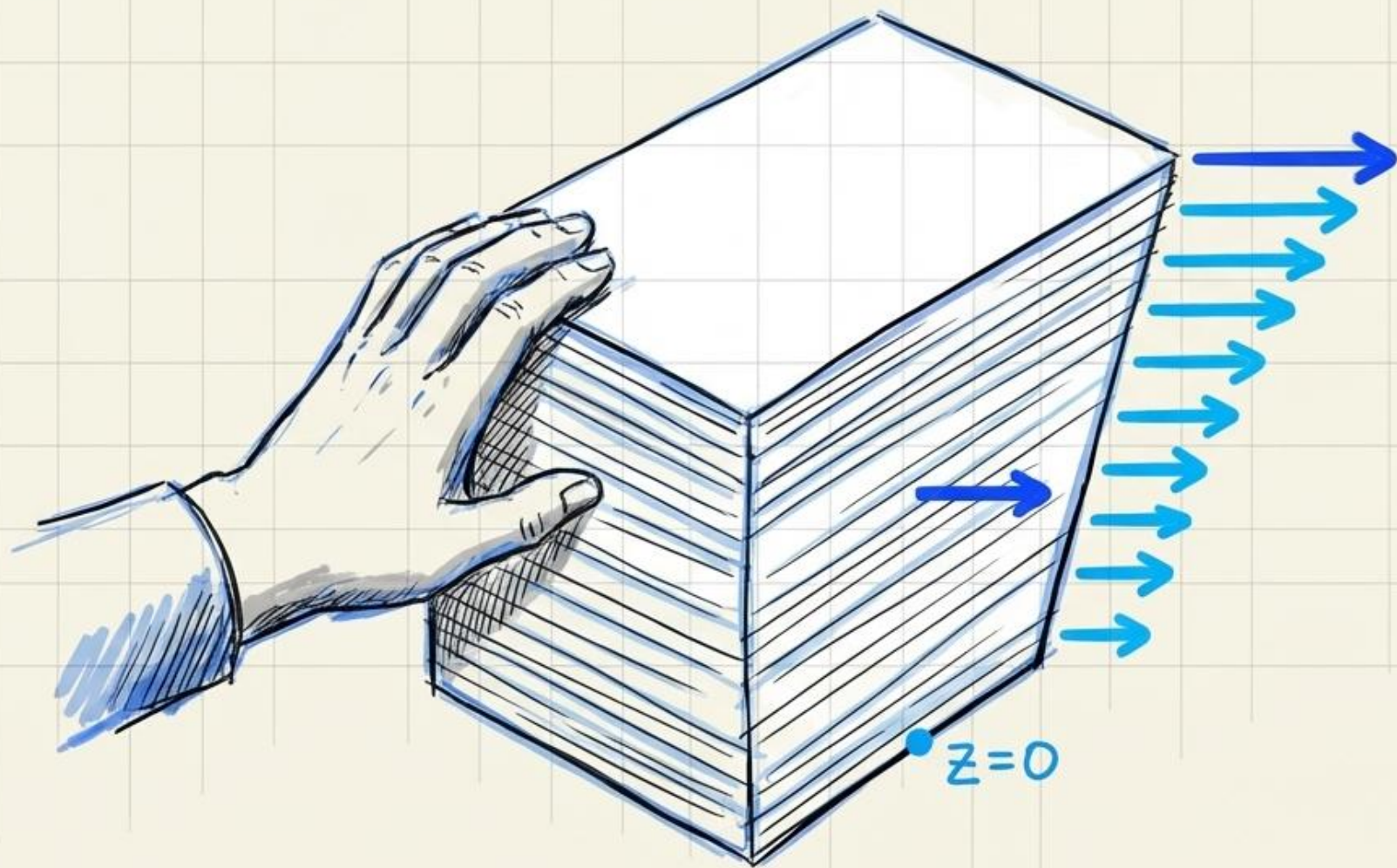
tip



→ 진짜 Translation입니다.



# 3x3 Lifted Transform: 종이 뭉치 비스듬하게 밀기

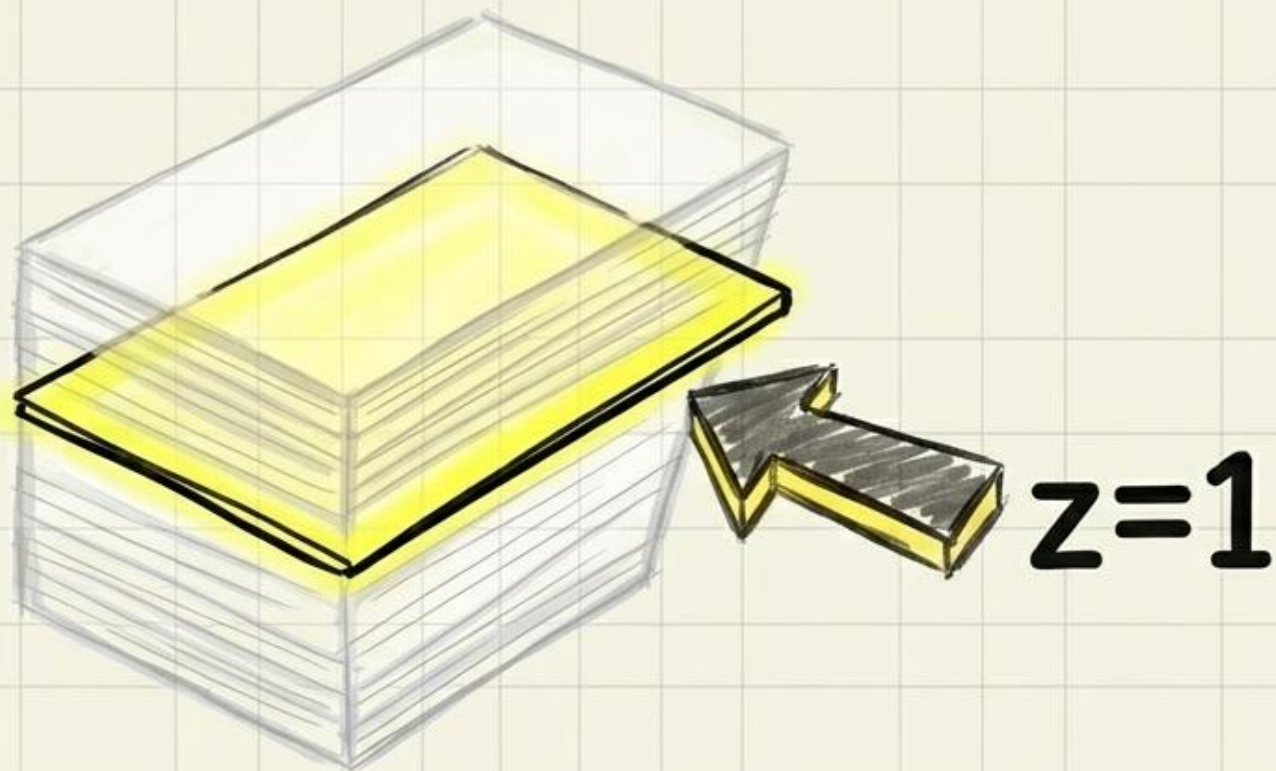


\* 3D 전체에서 보면  
Translation이 아니라  
Shear (기울임)입니다.

\*  $z$ 값에 비례하여 층마다  
이동량이 다릅니다  
( $z=0$ 은 이동 없음).



# 특정 슬라이스( $z=1$ )의 착시



- \* 오직  $z=1$  평면에 제한했을 때만 2D Translation처럼 보입니다.
- \* 특정 슬라이스에서는 같아 보이지만, 전체 공간 확장 방식이 완전히 다릅니다.



# 핵심 결론 (The Ultimate Takeaway)

Homogeneous Coordinate의 추가 차원( $w$ )은  
물리적 공간 축이 아닙니다.

Affine transform을 Linear algebra로 다루기 위해  
임시로 만든 임베딩(Embedding) 공간입니다.



# 1분 요약 Cheat Sheet



$[ ] * [ ]$

\* 목적: Translation을  
행렬곱(\*)으로 통합!

Point



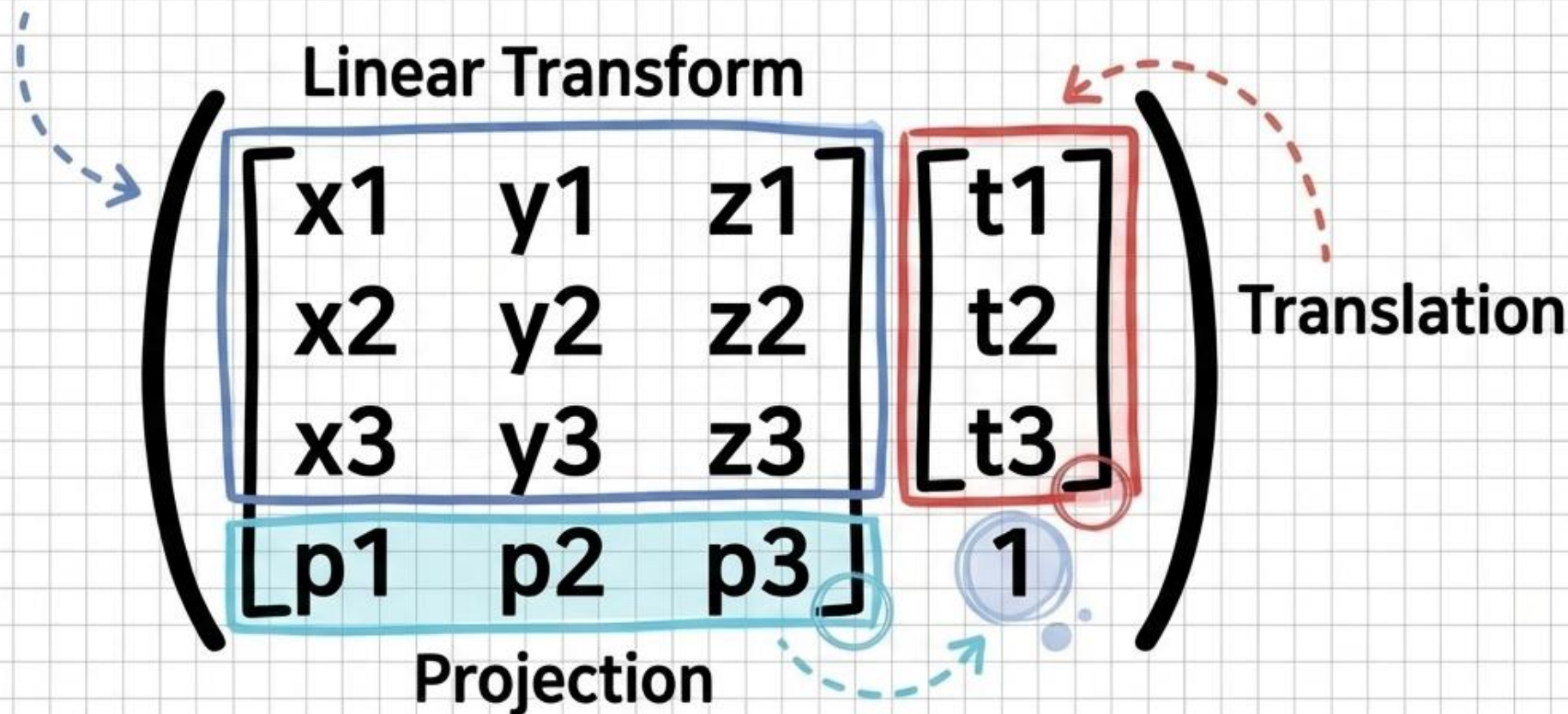
Vector

\* 규칙: Point는  $w=1$ ,  
Vector는  $w=0$



\* 본질: 4D 공간이 아니라  
수학적 임베딩(Embedding)!

# 마법의 4x4 매트릭스



이제 모든 Euclidean Transform을 단 하나의 4x4 매트릭스  
곱셈으로 통일할 수 있습니다!

그래픽스에서 가장 기본이 되는 궁극의 포맷입니다.



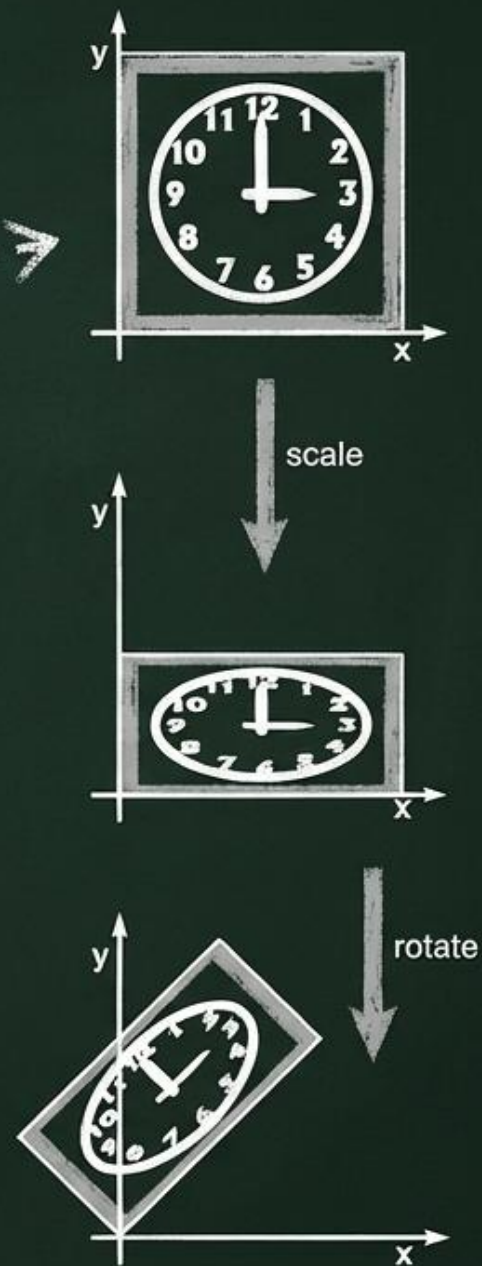
# 연속기 사용하기 (Composition)

$$v3 = \underline{R(S * v1)} \longrightarrow \textcircled{M} = \underline{R * S}$$

여러 개의 Transform을 연달아 적용해야 한다면?

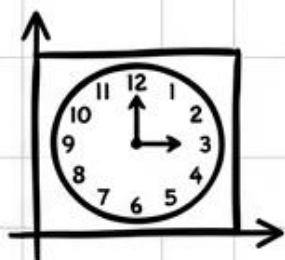
결합법칙을 이용해 매트릭스들을 미리 하나로 곱해둘 수 있습니다.

단 하나의 매트릭스 M으로 모든 움직임 처리 완료!



# Warning: Order Matters!

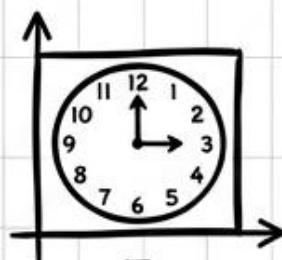
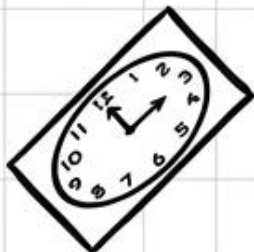
Transform Order: 곱하는 순서에 따라 결과가 완전히 달라집니다.  
원하는 결과를 얻으려면 순서 배치가 핵심입니다.



↓ scale



↓ rotate



↓ rotate

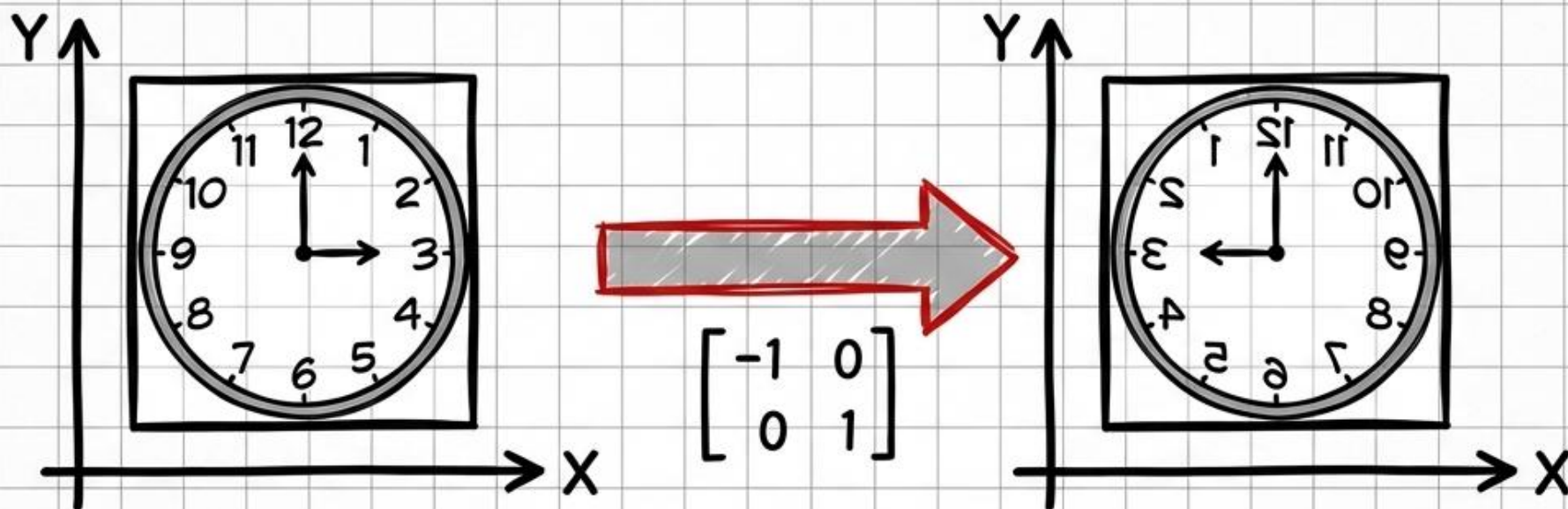


↓ scale





# Special Move: Reflection



Reflection: Scaling 값을 음수(-)로 설정하면 거울에 비친 것처럼 반전됩니다!

\*주의: Local Coordinate System의 형태(RHS에서 LHS로)가 바뀔 수 있습니다.

# Transform Cheat Sheet

오늘 배운 그래픽스 필수 생존 가이드 (Save this slide!)

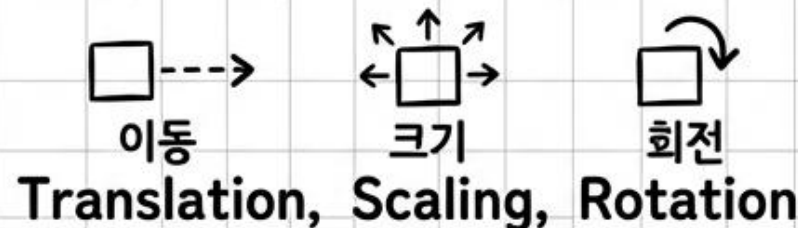
## 1. Point와 Vector의 차이 기억하기



$$\mathbf{P} - \mathbf{P} = \mathbf{V}$$

$$\mathbf{P} + \mathbf{V} = \mathbf{P}$$

## 2. 이동, 크기, 회전의 기본기 익히기



## 3. 4x4 Homogeneous Matrix로 모든 것을 하나로 통일하기!





**Questions? 😊**